

PROJECT DEMONSTRATION PHASE

AI FAQ ASSISTANT AI – AI-POWERED FAQ AND CUSTOMER SUPPORT AUTOMATION

1. INTRODUCTION

The AI FAQ Assistant is an AI-powered backend application developed to automate and simplify the process of creating, managing, searching, and maintaining frequently asked questions for customer support environments. The system is designed as a secure REST API platform that provides structured services for user authentication, FAQ management, category organization, semantic search, and AI-assisted content generation. The application is developed using Node.js and Express.js, with MongoDB serving as the primary database and Mongoose ODM being used for database modelling and schema validation. The overall system is designed to provide a centralized backend layer that can be accessed through API clients such as Postman or ThunderClient.

The main purpose of the application is to reduce the manual effort involved in preparing customer-support documentation. In a conventional customer-support environment, employees may have to repeatedly write answers to similar questions, organize those questions into categories, maintain existing documentation, and search through large amounts of information to find suitable answers. The AI FAQ Assistant addresses these requirements by providing a centralized platform where FAQ records can be created, searched, modified, and managed through RESTful APIs. The integration of Google Gemini AI further supports automated generation of FAQ questions, answers, and category suggestions based on a topic supplied by the user.

Security is an important part of the application. The system uses JSON Web Tokens for stateless user authentication and route protection. Protected resources require valid authentication information before the requested operation is executed. Passwords are protected using bcrypt hashing rather than being stored directly in their original form. The application also includes centralized error handling and schema validation so that invalid requests can be handled in a structured manner. These mechanisms together provide a controlled environment for managing user accounts and FAQ information.

The project demonstration phase focuses on showing the actual working flow of the developed application. Instead of describing only the design, the demonstration verifies the major operations through API requests and responses. User registration, user login, authentication, FAQ creation, FAQ search, and AI-based FAQ generation are demonstrated through the configured API endpoints. The demonstration also explains how the backend components communicate with the database and how authentication controls access to private operations.

2. OBJECTIVE OF THE PROJECT DEMONSTRATION

The objective of the project demonstration is to verify the functionality of the implemented AI FAQ Assistant backend and to show how the different components operate together as a complete system. The demonstration begins with the creation of a user account and continues with user authentication. Once the user has successfully logged in, the generated authentication information is used to access protected FAQ operations. This demonstrates the role of JWT authentication in controlling access to private resources.

The demonstration also verifies the FAQ management functionality by creating a new FAQ through a protected API endpoint. The created information is processed by the backend and stored using the MongoDB database through Mongoose. The search functionality is then demonstrated using a keyword-based request to retrieve relevant FAQ records. Finally, the AI generation feature is demonstrated by providing a topic to the AI generation endpoint and observing the FAQ information generated through the Google Gemini integration.

The demonstration therefore provides evidence that the system is not limited to a static interface or database structure but contains connected backend components that process requests, validate information, apply authentication, communicate with the database, and return structured responses to the API client.

3. SYSTEM ARCHITECTURE

The AI FAQ Assistant follows a layered server architecture that separates different responsibilities of the application. The architecture begins with an API interface through which requests are sent to the backend. Tools such as Postman and ThunderClient can be used to communicate with the API endpoints and observe the responses returned by the server. When a request reaches the application, it is received by the Express Server Gateway. The gateway is responsible for handling incoming HTTP requests and routing them to the appropriate operation.

The Express Server Gateway supports standard HTTP methods including GET, POST, PUT, and DELETE. It also processes JSON request bodies and provides the required server-level configurations. After receiving a request, the application determines whether the requested resource is public or protected. For protected operations, the authentication middleware is applied before the request reaches the relevant application logic.

The Authentication Middleware is responsible for extracting authentication information from the request headers and validating the JWT token. When a valid token is provided, the user's identity can be verified and the request can continue to the protected operation. If the required authentication information is not valid, the protected resource cannot be accessed through the normal authenticated route. This separation allows public operations such as registration and login to remain accessible while private operations such as FAQ creation and AI FAQ generation require authentication.

The application logic is organized into separate modules so that different responsibilities can be managed independently. The controller and service-level operations process the received request, apply the required rules, and communicate with the database or AI service. The database interface is implemented using Mongoose ODM, which translates application-level operations into MongoDB database operations. For AI-related requests, the application communicates with Google Gemini AI to generate content based on the topic provided by the user.

4. APPLICATION COMPONENTS

The API interface forms the communication layer between the client and the backend. During the demonstration, Postman or ThunderClient is used to send requests to the available endpoints. The request contains information such as the HTTP method, endpoint URL, request headers, and request body. After processing the request, the backend returns a JSON response that can be viewed directly in the API testing tool.

The Express Server Gateway acts as the central entry point for API communication. It receives the incoming request and determines which route should process it. This component allows the application to provide different operations through separate endpoints while maintaining a common server environment. The gateway also handles JSON request processing and other required Express configurations.

The authentication middleware provides the security layer for protected resources. It checks the JWT information supplied with the request and validates the authentication context before allowing the request to proceed. This mechanism ensures that private FAQ operations and AI-related operations are accessed through authenticated requests.

The database interface uses Mongoose ODM to communicate with MongoDB. Mongoose provides structured schemas for the application's entities and applies validation rules before information is stored. This helps maintain consistency in the database and prevents invalid information from being persisted through normal application operations.

The AI service integration connects the application to Google Gemini AI. When an authenticated user provides a topic through the AI generation endpoint, the backend processes the request and uses the AI service to generate FAQ-related content. The generated result can contain a question, answer, and category based on the supplied topic.

5. DATABASE IMPLEMENTATION

The AI FAQ Assistant uses MongoDB as its primary data storage system. MongoDB is used to maintain application information in document form, while Mongoose ODM provides the structure and validation required by the application. The database contains logical entities associated with users, FAQs, AI-generated information, and categories.

The User entity represents registered users of the application. It contains an ObjectId as the primary identifier along with the user's name, email address, password, role, creation time, and update time. The email field is required to be unique, while the password is stored in hashed form using bcrypt. The role field allows the application to distinguish different types of users such as administrators, content creators, and authenticated users.

The FAQ entity represents the main content managed by the application. Each FAQ contains an identifier, question, answer, category, creator reference, creation time, and update time. The createdBy field provides the logical connection between an FAQ and the user who created it. This relationship allows the application to associate FAQ records with their respective creators.

The AI Generation entity represents information produced dynamically by the Google Gemini AI service. It contains the topic supplied for generation along with the generated question, generated answer, generated category, and generation time. This entity is described as a logical entity because the generated information can be produced dynamically before being persisted or used by the application.

The Category entity is used to organize FAQ information. It contains a category name, description, and total FAQ count. Categories allow related FAQ records to be organized under a common classification, making the information easier to manage and retrieve.

6. USER ROLES AND ACCESS

The application defines different roles based on the type of activity a user is expected to perform. An administrator has access to complete system functionality and can manage registered users, FAQ records, AI-generated content, and FAQ categories. This role represents the management-level access required for maintaining the overall system.

A content creator is responsible for preparing and maintaining FAQ information. A content creator can create new FAQ records, modify personal FAQ records, remove owned records, generate FAQs using AI, and organize FAQ information under appropriate categories. This role is directly associated with the content management capabilities of the application.

An authenticated user can view available FAQ information, search for FAQs using keywords, generate AI-assisted answers, and access personal profile information. Authentication is required for operations that are protected by the application.

A public user has access to published FAQ information and can perform FAQ searches. However, public users cannot create, modify, or delete FAQ records and do not have access to protected API endpoints. This separation between public and protected operations provides a clear access structure within the application.

7. PROJECT SETUP AND CONFIGURATION

The project is configured as a Node.js application with the source code organized inside a dedicated source directory. The project is opened using Visual Studio Code or a similar code editor, allowing the different backend components to be developed and maintained separately. The application requires Node.js version 18 or above and npm version 9 or above according to the specified software requirements.

The initial Node.js project configuration is performed using the `npm init -y` command. The required application dependencies are installed using the npm package manager. The main dependencies include Express for server development, Mongoose for MongoDB interaction, dotenv for environment configuration, CORS for cross-origin request handling, jsonwebtoken for authentication, bcrypt for password hashing, and the Google Generative AI package for AI integration.

The project structure separates the server entry point, Express configuration, middleware, database models, and API routes. This modular structure allows individual parts of the application to be maintained without placing all functionality into a single source file. The server file provides the main application entry point, while the application configuration file manages Express-related setup. Middleware files provide authentication and centralized error handling, model files define database structures, and route files expose the required API operations.

8. BACKEND IMPLEMENTATION

The backend implementation begins with the gateway server, which acts as the main entry point of the application. It loads the required configuration, establishes the connection to MongoDB, and starts the server on the configured environment port. The Express application is then configured to handle incoming requests and connect them to the appropriate routes.

The authentication middleware forms an important part of the backend implementation. When a protected API request is received, the middleware extracts the JWT from the request headers and validates the token. The authenticated user context is then made available

to the relevant operation. This approach allows private routes to verify the identity of the requester before performing operations involving FAQ records or AI services.

The centralized error middleware provides a common mechanism for processing application errors. Instead of allowing individual routes to return inconsistent error structures, the middleware can process errors in a standardized manner. This improves the consistency of API responses and makes the backend easier to test and debug.

The Mongoose models provide the database-level structure of the application. The User model defines the structure and validation requirements for user information, while the FAQ model defines the fields required for FAQ records and their relationship with the user who created them.

9. API TESTING AND DEMONSTRATION

The practical demonstration of the project is performed using Postman or the ThunderClient extension available in Visual Studio Code. These tools allow API requests to be created and sent directly to the running backend application. The response from each endpoint can then be examined to verify whether the expected operation has been completed.

The first API demonstrated is the user registration operation using POST `/api/auth/register`. This endpoint is publicly accessible because a user must be able to create an account before authentication can take place. The request contains the user's name, email address, and password. In the demonstration, the sample user is Priya Sharma with the email address `priya@writeflow.com`. After the request is submitted, the backend processes the information and creates the user record according to the defined User schema.

The second API demonstrated is the login operation using POST `/api/auth/login`. The user supplies the registered email address and password. The backend verifies the credentials and provides JWT authentication information when the credentials are valid. This token is important for subsequent protected requests because it allows the backend to identify and authenticate the requesting user.

The third demonstration is FAQ creation through POST `/api/faqs`. This endpoint is protected and requires a Bearer JWT token. The request contains a question, answer, and category. The backend validates the authentication information and then processes the FAQ request. The resulting FAQ information is stored through the Mongoose database layer and associated with the authenticated user.

10. FAQ SEARCH DEMONSTRATION

The FAQ search operation is demonstrated through the endpoint GET `/api/faqs/search?q=configure`. Unlike the protected FAQ creation endpoint, the search operation is publicly accessible according to the project specification. The keyword `configure` is supplied as a query parameter. The backend processes the query and searches the available FAQ information for relevant matching content.

The search functionality provides a way to retrieve useful information without manually examining every FAQ record. This is particularly relevant to the project's customer-support scenario, where users may need to locate an answer quickly. The returned JSON response

can be inspected in Postman or ThunderClient to verify that the search operation is functioning correctly.

11. AI FAQ GENERATION DEMONSTRATION

The AI FAQ generation operation demonstrates the integration between the backend application and Google Gemini AI. The endpoint used for this operation is POST /api/ai/generate-faq, which is protected and therefore requires a valid Bearer JWT token.

During the demonstration, the authenticated user provides the topic “Mongoose schema indexing validation runtime workflow optimization.” This topic is submitted as a JSON request to the API. The backend receives the request and passes the relevant information to the Google Gemini AI integration. The AI service processes the topic and produces FAQ-related content.

The generated result can include a question, answer, and category. This demonstrates how the application can automate part of the FAQ creation process instead of requiring the content creator to manually prepare every question and answer. The AI generation functionality therefore supports the main objective of reducing repetitive manual work in customer-support documentation.

12. SECURITY AND VALIDATION DURING DEMONSTRATION

Security mechanisms are demonstrated throughout the API workflow. The registration process protects passwords through bcrypt hashing before they are stored. During login, the backend verifies the submitted credentials and generates JWT authentication information. Protected endpoints require this authentication information before the requested operation can be performed.

The FAQ creation and AI generation endpoints demonstrate the protected nature of application operations. A valid Bearer token is required to access these resources. The authentication middleware validates the token before the request reaches the relevant application logic. This provides a controlled mechanism for restricting access to private resources.

Mongoose schema validation also contributes to the reliability of the application. The defined User and FAQ schemas specify required fields and data structures. Before information is stored in MongoDB, it is processed according to the defined schema rules. This helps maintain the structural integrity of the stored information.

Centralized error handling provides an additional layer of reliability. When an invalid request or application problem occurs, the error-handling middleware can provide a standardized response instead of allowing unhandled errors to propagate through the application. This makes the API easier to test and maintain.

13. DEMONSTRATION RESULTS

The project demonstration covers the major functions defined in the AI FAQ Assistant. The user registration process demonstrates the creation of a new application account. The login operation demonstrates authentication and JWT generation. The protected FAQ creation operation demonstrates authenticated access to content management functionality. The search operation demonstrates retrieval of relevant FAQ information through a keyword. The AI generation

operation demonstrates the integration of the backend with Google Gemini AI for automated FAQ content generation.

The complete demonstration therefore follows a connected workflow beginning with user onboarding and authentication and continuing through content management, search, and AI-assisted generation. The MongoDB database provides persistent storage for the application's structured information, while Mongoose provides the required data modelling and validation. The API layer provides a common interface through which these operations can be tested and verified.

14. CONCLUSION

The project demonstration of the AI FAQ Assistant verifies the major functionality of the developed REST API platform. The demonstration begins with user registration and authentication and proceeds to protected FAQ creation, FAQ searching, and AI-powered FAQ generation. The successful interaction between the Express.js server, authentication middleware, Mongoose database layer, MongoDB database, and Google Gemini AI integration demonstrates the working structure of the application.

The use of JWT provides authentication for protected API operations, while bcrypt provides password hashing. Mongoose schema validation helps maintain structured database information, and centralized error handling provides consistent API behaviour. The FAQ management and search functionalities support the organization and retrieval of customer-support information, while the AI generation functionality reduces the manual effort involved in preparing FAQ content.

Overall, the AI FAQ Assistant demonstrates a complete backend workflow for AI-powered FAQ and customer-support automation. The project provides the required foundation for managing users, FAQ records, categories, search operations, and AI-generated content through a secure and structured REST API architecture.